# Colecciones: Pilas, Colas y Bolsas

## Algoritmos y Estructuras de Datos

Rodrigo Pérez Rodríguez

Universidad Rey Juan Carlos

# Contenido


1. Introducción
2. APIs de pilas, colas y bolsas
3. Genéricos
4. Colecciones iterables
5. Bolsas
6. Colas
7. Pilas
8. Implementación de colecciones
9. Listas enlazadas
10. Discusión

# Introducción

# Pilas, Colas y Bolsas

**Tres tipos de estructuras de datos básicas para almacenar colecciones de valores**

- Se utilizan directamente en aplicaciones
- Son bloques constructivos de estructuras más complejas
- API incluye operaciones para añadir, extraer y consultar valores
- Un mismo valor puede estar almacenado varias veces
- **Diferencia principal:** política de extracción de elementos

# Más allá de los arrays

**Limitaciones de los arrays:**

- Almacenamiento contiguo en memoria
- Capacidad fija una vez creados

**TADs para colecciones:**

- Formas alternativas de acceso a los datos
- Ejemplo: leer el último dato almacenado, independientemente de su posición

# APIs de pilas, colas y bolsas

# Características comunes de las APIs

**Todas las APIs contienen:**

- Constructor sin argumentos
- Método para añadir elementos a la colección
- Método para comprobar si está vacía
- Método que devuelve el tamaño de la colección

**Pilas y colas además tienen:**

- Método para extraer un objeto de la colección
- **Pila:** extrae el último añadido (LIFO)
- **Cola:** extrae el primero añadido (FIFO)

# API de la Bolsa

```
public class Bag<Item> implements Iterable<Item>

    Bag()                    // constructor: crea una nueva bolsa
    void add(Item item)      // añade un elemento (ítem)
    boolean isEmpty()        // ¿está vacía la bolsa?
    int size()               // número de ítems en la bolsa
```

**Características:**

- No hay operación para extraer elementos
- El iterador permite consultar todos los elementos
- El orden de iteración no está especificado

# API de la Cola

```
public class Queue<Item> implements Iterable<Item>

    Queue()                  // constructor: crea una nueva cola
    void enqueue(Item item)  // añade un elemento (ítem)
    Item dequeue()           // extrae y devuelve el ítem que más
                             // tiempo lleva almacenado (FIFO)
    boolean isEmpty()        // ¿está vacía la cola?
    int size()               // número de ítems en la cola
```

**Política FIFO (First-In-First-Out):**

- Se extrae el elemento que más tiempo lleva almacenado
- Como las colas en la vida cotidiana

# API de la Pila

```
public class Stack<Item> implements Iterable<Item>

    Stack()                 // constructor: crea una nueva pila
    void push(Item item)    // añade un elemento (ítem)
    Item pop()              // extrae y devuelve el ítem más
                            // recientemente añadido (LIFO)
    boolean isEmpty()       // ¿está vacía la pila?
    int size()              // número de ítems en la pila
```

**Política LIFO (Last-In-First-Out):**

- Se extrae el último elemento añadido
- Como una pila de libros o platos

# Importancia de los comentarios en las APIs

**Pila vs Cola:**

- APIs idénticas salvo por nombres de clase y métodos
- Sin leer los comentarios, no se distinguirían
- **Los comentarios forman parte de la especificación**
- El código no da información sobre las diferencias de comportamiento

**Conclusión**

La documentación es esencial para especificar el comportamiento de los TADs

# Genéricos

# El problema de los tipos

**Necesidad de diferentes tipos:**

- Aplicaciones diferentes necesitan pilas de diferentes tipos
- Pilas de String, int, Transaction, etc.
- ¿Cómo evitar implementar una clase para cada tipo?

**Soluciones tradicionales:**

- **Lenguajes dinámicos (Python):** cualquier tipo en cualquier variable
  - Ventaja: flexibilidad
  - Desventaja: errores en tiempo de ejecución
- **Lenguajes estrictos sin genéricos:** conversiones inseguras
  - Ventaja: errores en compilación
  - Desventaja: código repetido o inseguro

# Genéricos: lo mejor de dos mundos

**Ventajas de los genéricos:**

- Una sola implementación para todos los tipos
- El compilador detecta errores de tipo en tiempo de compilación
- Código más robusto y seguro
- No hay penalización en rendimiento

## Ejemplo

`Stack<String>` es una pila de String
`Stack<Integer>` es una pila de Integer
Una sola implementación: `Stack<Item>`

# Parámetros genéricos de tipo

**Sintaxis:** `Stack<Item>`

- `<Item>` es un parámetro genérico de tipo
- `Item` actúa como comodín para el tipo de elementos
- No es un tipo concreto, es un marcador de posición
- El código de la clase puede referirse a `Item`

**Instanciación:**

```
Stack<String> stack = new Stack<String>();
```

- El cliente sustituye `Item` por `String`
- Crea una pila específica para String

# Ejemplo: Stack de String

```
Stack<String> stack = new Stack<String>();
stack.push("Hello");
stack.push("World");
String next = stack.pop();  // "World"
```

**Error de compilación:**

```
stack.push(new Date(12, 31, 1999));
// Error: incompatible types
```

El compilador detecta que intentamos añadir un Date a una pila de String

# Ejemplo: Queue de Date

```
Queue<Date> queue = new Queue<Date>();
queue.enqueue(new Date(12, 31, 1999));
queue.enqueue(new Date(1, 1, 2000));
Date next = queue.dequeue();  // Date(12, 31, 1999)
```

**Error de compilación:**

```
queue.enqueue("Test");
// Error: incompatible types
```

El compilador detecta que intentamos añadir un String a una cola de Date

# Autoboxing y auto-unboxing

**Tipos primitivos con genéricos:**

- Java convierte automáticamente tipos primitivos a wrappers
- Y viceversa (auto-unboxing)

**Ejemplo:**

```
Stack<Integer> stack = new Stack<Integer>();
stack.push(17);      // autoboxing: int -> Integer
int i = stack.pop();   // unboxing: Integer -> int
```

Se aplica en:

- Asignaciones
- Paso de parámetros
- Expresiones aritmético-lógicas

# Colecciones iterables

# ¿Por qué iteradores?

**Necesidad:**

- A veces queremos consultar todos los elementos (sin extraerlos)
- Independientemente de la política de extracción
- Sin extraerlos de la colección

**Caso especial - Bolsas:**

- No tienen operación de extracción
- Sin iterador, serían casi inútiles
- El iterador permite consultar todos los valores

# Sentencia foreach

```
Queue<Transaction> collection = new Queue<Transaction>();
// ... añadir transacciones ...

for (Transaction transaction : collection)
{
    System.out.println(transaction);
}
```

**Características:**

- Sintaxis simple y clara
- Funciona con Stack, Queue y Bag
- No extrae elementos, solo los consulta
- Requiere que la colección implemente `Iterable<Item>`

# Foreach vs Iterator

**El foreach es azúcar sintáctico:**

```
// Sentencia foreach
for (Transaction t : collection)
    System.out.println(t);

// Es equivalente a:
Iterator<Transaction> i = collection.iterator();
while (i.hasNext())
{
    Transaction t = i.next();
    System.out.println(t);
}
```

**Ventaja de los iteradores:**

- El código cliente no necesita conocer la implementación interna
- Misma interfaz para diferentes estructuras de datos

# Bolsas

# Concepto de Bolsa

**Analogía:** Bolsa física para guardar objetos

- No importa el orden de almacenamiento
- Solo podemos añadir elementos
- No podemos extraer elementos
- Podemos iterar sobre todos ellos

**Orden de iteración:**

- No está especificado
- El código cliente no debe depender del orden

# Ejemplo: Bolsa de bolas de colores

**Escenario:**

- Guardamos bolas de diferentes colores
- No importa en qué orden se añadieron
- Queremos contar cuántas hay de cada color

**Código cliente:**

- Recorre todas las bolas con foreach
- Cuenta las de cada color
- Cualquier orden de iteración es válido
- El resultado es el mismo independientemente del orden

# Ejemplo: Stats.java

```java
import edu.princeton.cs.algs4.Bag;
import edu.princeton.cs.algs4.StdIn;

public class Stats {
    public static void main(String[] args) {
        Bag<Double> numbers = new Bag<Double>();
        while (!StdIn.isEmpty())
            numbers.add(StdIn.readDouble());

        int n = numbers.size();
        double sum = 0.0;
        for (double x : numbers)
            sum += x;
        double mean = sum/n;

        sum = 0.0;
        for (double x : numbers)
            sum += (x - mean)*(x - mean);
        double stddev = Math.sqrt(sum/(n-1));

        System.out.printf("Mean: %.2f\n", mean);
        System.out.printf("Stddev: %.2f\n", stddev);
    }
}
```

# Ejecución de Stats.java

**Entrada:**

- 100, 99, 101, 120, 98, 107, 109, 81, 101, 90

**Salida:**

- Mean: 100.60
- Stddev: 10.51

**Observaciones:**

- El orden de los números no afecta al resultado
- La bolsa permite procesar todos los valores
- No necesitamos extraer elementos

# Colas

# Concepto de Cola

**Ejemplos de la vida cotidiana:**

- Fila en una taquilla de cine
- Atasco en una carretera
- Cola de impresión compartida
- Secuencia de acciones de un robot

**Política FIFO (First-In-First-Out):**

- Se extrae el elemento que más tiempo lleva almacenado
- Mantiene el orden de inserción
- La persona que lleva más tiempo esperando es atendida primero

# Operación dequeue()

**Diferencia con Stack y Bag:**

- Queue tiene operación `dequeue()`
- Extrae el elemento más antiguo (FIFO)
- Stack: extrae el más reciente (LIFO)
- Bag: no tiene extracción

**Iteración:**

- El foreach recorre en orden de inserción
- Permite consultar sin extraer
- Útil cuando queremos mantener el orden

# Ejemplo: TestQueue.java

```java
import edu.princeton.cs.algs4.Queue;
import edu.princeton.cs.algs4.StdIn;

public class TestQueue {
    public static void main(String[] args) {
        Queue<Integer> q = new Queue<Integer>();

        while (!StdIn.isEmpty())
            q.enqueue(StdIn.readInt());

        int N = q.size();
        int[] a = new int[N];
        for (int i = 0; i < N; i++)
            a[i] = q.dequeue();

        for (int i = 0; i < N; i++)
            System.out.println(a[i]);
    }
}
```

**Uso:** Lee enteros, crea un array manteniendo el orden de entrada

# Pilas

# Concepto de Pila

**Ejemplos de la vida cotidiana:**

- Pila de libros o papeles sobre una mesa
- Ropa apilada en un estante
- Correo electrónico (el último leído primero)
- Historial de páginas web en un navegador

**Política LIFO (Last-In-First-Out):**

- Se extrae el último elemento añadido
- El elemento en la cima de la pila
- Invierte el orden de inserción

# Operación pop()

**Diferencia con Queue:**

- Stack tiene operación `pop()`
- Extrae el elemento más reciente (LIFO)
- Queue: extrae el más antiguo (FIFO)

**Iteración:**

- El foreach recorre en orden inverso al de inserción
- Útil para dar la vuelta a una secuencia
- El más reciente se procesa primero

# Ejemplo: Reverse.java

```
import edu.princeton.cs.algs4.Stack;
import edu.princeton.cs.algs4.StdIn;

public class Reverse {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<Integer>();

        while (!StdIn.isEmpty())
            stack.push(StdIn.readInt());

        for (int i : stack)
            System.out.println(i);
    }
}
```

**Resultado:** Muestra los enteros en orden inverso al de entrada

# Algoritmo de Dijkstra: Introducción

**Problema:** Evaluar expresiones aritméticas

- Expresiones completamente parentizadas
- Cada operación entre paréntesis
- No hay precedencia de operadores
- El orden viene dado por los paréntesis

**Ejemplos:**

- `( ( 3 + 2 ) * 5 )` $= 25$
- `( 3 + ( 2 * 5 ) )` $= 13$
- `( ( ( ( 1 + sqrt 4 ) + 1 ) + ( ( 2 + 3 ) * ( 4 * 5 ) ) ) )`

# Definición de expresión aritmética

**Definición recursiva:**

1. Un número es una expresión aritmética
2. ( expresión operador expresión ) es una expresión aritmética

**Operadores soportados:**

- Infijos: `+ - * /`
- Unario: `sqrt` (raíz cuadrada)

**Requisitos:**

- Todas las expresiones entre paréntesis
- Números separados por espacios
- Caracteres separados por espacios

# Algoritmo de dos pilas de Dijkstra

**Dos pilas:**

1. Pila de operandos
2. Pila de operadores

**Algoritmo (izquierda a derecha):**

- Paréntesis abierto: ignorar
- Operador: push en pila de operadores
- Operando: push en pila de operandos
- Paréntesis cerrado:
  - Pop un operador
  - Pop los operandos necesarios
  - Aplicar operador a operandos
  - Push resultado en pila de operandos

# ¿Por qué funciona?

**Idea clave:**

- Cada subexpresión entre paréntesis se evalúa
- El resultado substituye a la subexpresión
- Es como si el valor hubiera aparecido directamente
- Al final queda un solo valor en la pila de operandos

**Ejemplo:** `( ( 3 + 2 ) * 5 )`

1. Al llegar a `)`, pop +, pop 2 y 3, push 5
2. Al llegar al segundo `)`, pop *, pop 5 y 5, push 25
3. Resultado final: 25

# Dijkstra.java (1/3)

```
import java.util.Scanner;

public class Dijkstra {
    private static boolean isBinOperator(String op) {
        String ops = "+-*/";
        return ops.contains(op);
    }

    private static double evalOp(Stack<Double> vals, double v1, String op) {
        double v = 0.0;
        double v2 = 0.0;
        if (isBinOperator(op)) {
            v2 = vals.pop();
        }
        if (op.equals("+")) {
            v = v1 + v2;
```

# Dijkstra.java (2/3)

```
        } else if (op.equals("-")) {
            v = v1 - v2;
        } else if (op.equals("*")) {
            v = v1 * v2;
        } else if (op.equals("/")) {
            v = v1 / v2;
        } else if (op.equals("sqrt")) {
            v = Math.sqrt(v1);
        }
        return v;
    }

    public static void main(String[] args) {
        Stack<String> ops = new Stack<String>();
        Stack<Double> vals = new Stack<Double>();
        Scanner sc = new Scanner(System.in);
```

# Dijkstra.java (3/3)

```
        // Lee toda la expresión en una sola línea
        String input = sc.nextLine();
        for(int i = 0; i < input.length(); i++){
            String s = "" + input.charAt(i);
            if (s.equals("(")) {
                // ignoramos
            } else if (isBinOperator(s) || s.equals("sqrt")) {
                ops.push(s);
            } else if (s.equals(")")) {
                String op = ops.pop();
                double v = vals.pop();
                v = evalOp(vals, v, op);
                vals.push(v);
            } else { // El token es un operando
                vals.push(Double.parseDouble(s));
            }
        }
        System.out.println(vals.pop());
    }
}
```

**Nota:** Lee toda la expresión en una línea (funciona con números de 1 dígito)

# Ejecución de Dijkstra.java

**Ejemplo 1:** `( ( 3 + 2 ) * 5 )`

- Resultado: 25.0

**Ejemplo 2:** `( 3 + ( 2 * 5 ) )`

- Resultado: 13.0

**Importancia:**

- Código sencillo pero potente
- Ilustra funcionamiento de intérpretes

# Ejemplo visual: ((3+2)*5)

| Token | Acción | Pila ops | Pila vals |
|---|---|---|---|
| ( | ignorar | — | — |
| ( | ignorar | — | — |
| 3 | push val | — | 3 |
| + | push op | + | 3 |
| 2 | push val | + | 3, 2 |
| ) | evaluar: 3+2=5 | — | 5 |
| * | push op | * | 5 |
| 5 | push val | * | 5, 5 |
| ) | evaluar: 5*5=25 | — | 25 |

**Resultado final:** 25

# Implementación de colecciones

# Estrategia de implementación

**Enfoque incremental:**

1. Pila de String con capacidad fija
2. Mejoras sucesivas:
   - Hacer genérica
   - Capacidad dinámica
   - Añadir soporte para iteradores
   - Implementar con listas enlazadas

# API: FixedCapacityStackOfStrings

| Método | Descripción |
|---|---|
| `FixedCapacityStackOfStrings(int cap)` | crea pila de capacidad cap |
| `void push(String item)` | añade un string |
| `String pop()` | extrae el más reciente (LIFO) |
| `boolean isEmpty()` | ¿está vacía? |
| `int size()` | número de strings |

**Limitaciones:**

- No es genérica: solo String
- Capacidad fija especificada en el constructor
- No soporta iteradores (no hay foreach)

# Implementación: FixedCapacityStackOfStrings (1/2)

```
import java.util.Scanner;

public class FixedCapacityStackOfStrings
{
    private String[] a;     // elementos de la pila
    private int n = 0;      // número de elementos en la pila

    public FixedCapacityStackOfStrings(int capacity)
    {  a = new String[capacity]; }

    public boolean isEmpty() { return n == 0; }
    public int size()        { return n; }

    public void push(String item) {
        a[n] = item;
        n++;
    }

    public String pop() {
        n--;
        return a[n];
        // return a[--n];
    }
```

# Implementación: FixedCapacityStackOfStrings (2/2)

```
    public static void main (String[] args)
    {
        FixedCapacityStackOfStrings s;
        s = new FixedCapacityStackOfStrings(100);
        Scanner sc = new Scanner(System.in);
        while (sc.hasNextLine()) {
            String item = sc.nextLine();
            if (!item.equals("-")) { s.push(item); }
            else if (!s.isEmpty()) { System.out.println(s.pop()); }
        }
        System.out.println("(" + s.size() + " elements on the stack)");

        int n = s.size();
        for (int i = 0; i < n; i++) {
            System.out.println(s.pop());
        }
    }
}
```

**Uso:** Lee líneas de entrada. Si es "–", extrae y muestra. Al final, muestra elementos restantes.

# Decisiones de representación

**Variables de instancia:**

- `a[]`: array de String
- `n`: número de valores almacenados

**Operaciones:**

- **Extraer:** decrementa `n`, devuelve `a[n]`
- **Insertar:** copia en `a[n]`, incrementa `n`

**Invariante:**

- El elemento en la cima está en `a[n-1]`
- Posiciones `0..n-1` contienen elementos
- Posiciones `n..length-1` están vacías

# Mejora 1: Pila genérica - FixedCapacityStack<Item>

**Problema:** FixedCapacityStackOfStrings solo almacena String
**Solución:** Usar genéricos

- Cambiar `String` por parámetro genérico `Item`
- Declarar: `public class FixedCapacityStack<Item>`
- Array: `private Item[] a;`

**Problema técnico de Java:**

- No se permite `a = new Item[capacity];`
- Tampoco se puede hacer `new Item()`: en tiempo de ejecución `Item` es sólo un `Object`, no se conoce el constructor específico
- Solución: `a = (Item[]) new Object[capacity];`
- Se crea un array de `Object` y se hace cast a `Item[]`

# Implementación: FixedCapacityStack<Item>

```
public class FixedCapacityStack<Item>
{
    private Item[] a;     // elementos de la pila
    private int n = 0;    // número de elementos

    public FixedCapacityStack(int capacity)
    {  a = (Item[]) new Object[capacity]; }

    public boolean isEmpty() { return n == 0; }
    public int size()        { return n; }

    public void push(Item item) {
        a[n++] = item;
    }

    public Item pop() {
        return a[--n];
    }
}
```

**Ventaja:** Una sola implementación para todos los tipos

# Mejora 2: Capacidad dinámica - ResizingArrayStack<Item>

**Problema:** Capacidad fija es limitante
**Solución:** Redimensionar el array dinámicamente
**Estrategia:**

- En `push()`: si el array está lleno, doblar su tamaño
- En `pop()`: si el array está 1/4 lleno, reducir a la mitad
- Usar método privado `resize(int max)`

**¿Por qué 1/4 y no 1/2?**

- Evitar thrashing: redimensionar constantemente
- Si reducimos al 50 %, añadir/quitar puede redimensionar siempre

# Método resize()

```
private void resize(int max)
{
    // Mover pila a un nuevo array de tamaño max
    Item[] temp = (Item[]) new Object[max];
    for (int i = 0; i < n; i++)
        temp[i] = a[i];
    a = temp;  // El array antiguo será recuperado por GC
}
```

## Uso en push():

```
public void push(Item item)
{
    if (n == a.length) resize(2*a.length);
    a[n++] = item;
}
```

# Uso de resize en pop()

```
public Item pop()
{
    Item item = a[--n];
    a[n] = null;  // Evitar loitering
    if (n > 0 && n == a.length/4)
        resize(a.length/2);
    return item;
}
```

**Loitering:**

- Asignar `null` libera la referencia
- Permite al recolector de basura recuperar memoria del objeto
- Sin esto, el objeto permanece en memoria innecesariamente

# Mejora 3: Soporte para iteración

**Objetivo:** Permitir usar foreach para recorrer la colección
**Código cliente deseado:**

- `for (String s : stack) { System.out.println(s); }`

**Ventajas de implementar** `Iterable<Item>`**:**

- Permite usar sintaxis foreach elegante y compacta
- Código cliente independiente de la implementación interna
- Oculta detalles de cómo se almacenan los elementos

# Interfaz Iterable: Requisitos

**Implementar `Iterable<Item>` obliga a:**

1. Declarar que la clase implementa la interfaz:
   `public class Stack<Item> implements Iterable<Item>`
2. **Implementar obligatoriamente el método:**
   `public Iterator<Item> iterator()`
   - Este método debe retornar un objeto de tipo `Iterator<Item>`
   - El compilador fuerza su implementación

**Implicación:**

- Necesitamos crear una clase que implemente `Iterator<Item>`
- Esta clase iteradora tiene sus propios métodos obligatorios

# Interfaz Iterator: Métodos a implementar

**Al implementar `Iterator<Item>`, el compilador obliga a implementar:**
**Métodos obligatorios:**

1. `public boolean hasNext()`
   - Retorna `true` si quedan elementos por iterar
   - Se llama antes de cada llamada a `next()`
2. `public Item next()`
   - Retorna el siguiente elemento de la colección
   - Avanza el iterador a la siguiente posición
   - Lanza excepción si no hay más elementos

**Método opcional (desde Java 8 tiene implementación por defecto):**

- `default void remove()` – Elimina el último elemento retornado
- Generalmente no se reimplementa (la versión por defecto lanza excepción)

# Implementación del iterador (1/2)

```
import java.util.Iterator;

public class ResizingArrayStack<Item> implements Iterable<Item>
{
    private Item[] a = (Item[]) new Object[1];
    private int n = 0;

    // ... métodos isEmpty(), size(), resize(), push(), pop() ...

    public Iterator<Item> iterator()
    {
        return new ReverseArrayIterator();
    }

    private class ReverseArrayIterator implements Iterator<Item>
    {
        private int i = n-1;  // posición actual del iterador
```

# Implementación del iterador (2/2)

```
        public boolean hasNext()
        {
            return i >= 0;
        }

        public Item next()
        {
            return a[i--];
        }

        public void remove() { }  // No implementado
    }
}
```

**Funcionamiento:**

- Itera en orden LIFO (último añadido primero)
- Variable `i` apunta al siguiente elemento a devolver
- `hasNext()`: comprueba si quedan elementos
- `next()`: devuelve elemento y avanza

# ResizingArrayStack<Item>: Código completo (1/3)

```
import java.util.Iterator;
import java.util.Scanner;

public class ResizingArrayStack<Item> implements Iterable<Item>
{
    // Inicialmente, el tamaño del array es 1
    private Item[] a; // Tenemos que hacer un cast a Item[] para que encajen los tipos
    private int n;

    public ResizingArrayStack() {
        a = (Item[]) new Object[1];
        n = 0;
    }

    public boolean isEmpty() {
        return n == 0;
    }
    public int size() {
        return n;
    }

    private void resize(int max) {   // Mover pila a un nuevo array temp, de tamaño max
        Item[] temp = (Item[]) new Object[max];
        for (int i = 0; i < n; i++)  {
            temp[i] = a[i];
        }
        a = temp;
    }
```

# ResizingArrayStack<Item>: Código completo (2/4)

```
public void push(Item item) {   // Añadir item a la cima de la pila
    if (n == a.length) {
        resize(2*a.length);
    }
    a[n++] = item; // primero se añade, luego se incrementa
}

public Item pop() {   // Extraer item de la cima de la pila y devolverlo
    Item item = a[--n];
    a[n] = null;  // Evitar loitering. Si no se hace, el objeto no se puede
                  // recoger, ya que alguien lo sigue referenciando
    if (n > 0 && n == a.length/4) {
        resize(a.length/2);
    }
    return item;
}
```

# ResizingArrayStack<Item>: Código completo (3/4)

```
public Iterator<Item> iterator() {
    return new ReverseArrayIterator();
}

private class ReverseArrayIterator implements Iterator<Item> {
    private int i = n-1; // posición del iterador

    public boolean hasNext()  {
        return i >= 0;
    }
    public Item next()  {
        return a[i--];
    }
    public void remove() {
    }
}
```

# ResizingArrayStack<Item>: Código completo (4/4)

```
    public static void main (String[] args) {

        ResizingArrayStack<String> stack_of_strings;
        stack_of_strings = new ResizingArrayStack();
        Scanner sc = new Scanner(System.in);
        while (sc.hasNext())
        {
            String item = sc.next();
            if (!item.equals("-")) { stack_of_strings.push(item); }
            else if (!stack_of_strings.isEmpty()) { System.out.print(stack_of_strings.pop() + " "); }
        }
        System.out.println("(" + stack_of_strings.size() + " left on stack)");

        // recorremos la pila con el iterador para los elementos que quedan
        for(String aux : stack_of_strings){
            System.out.println(aux);
        }

    }
}
```

# ResizingArrayStack<Item>: Características finales y rendimiento

**Características finales:**

- ✓ Genérica (funciona con cualquier tipo)
- ✓ Capacidad dinámica (crece y se reduce)
- ✓ Iterable (soporta foreach)
- ✓ Previene loitering

**Rendimiento:**

- Casi todas las operaciones en tiempo constante
- Redimensionar: tiempo proporcional a N
- Pero ocurre raramente (análisis amortizado)
- Espacio proporcional al tamaño de la colección

# Listas enlazadas

# Motivación

**Problema pendiente con arrays:**

- Redimensionar requiere copiar todos los elementos
- Operación costosa (proporcional a N)

**Solución: Listas enlazadas**

- Estructura de datos alternativa
- Tamaño dinámico sin copiar elementos
- Inserción/eliminación en tiempo constante

**Concepto:**

- Nodos que almacenan valores
- Enlaces (referencias) entre nodos
- Crecimiento/reducción sin redimensionar

# Clase Node

```
private class Node
{
    Item item;
    Node next;
}
```

**Definición recursiva de lista enlazada:**

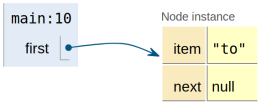
- O bien está vacía (`null`)
- O bien es un nodo que contiene:
  - Un valor (`Item`)
  - Una referencia a otra lista enlazada (`next`)

**Características:**

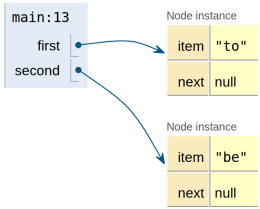
- **No es un TAD:** es una clase auxiliar para almacenar valores dentro de la colección
- Clase anidada (inner class) dentro de Stack/Queue/Bag
- `private`: oculta detalles de implementación al cliente
- Sin métodos: solo almacena datos (estructura de datos pasiva)

# Construyendo una lista enlazada: Ejemplo visual

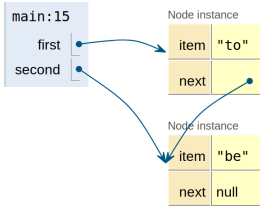
```
Node first = new Node();
first.item = "to";
```



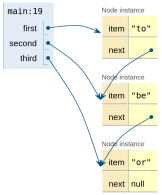

```
Node second = new Node();
second.item = "be";
```




```
first.next = second;
```




```
Node third = new Node();
third.item = "or";
second.next = third;
```



# Recorrer una lista enlazada

**Comparación con arrays:**

Para recorrer un array:

```
for (int i = 0; i < n; i++) {
    // procesamos a[i]
}
```

**Para recorrer una lista enlazada con `for`:**

```
for (Node x = first; x != null; x = x.next) {
    // procesamos x.item
}
```

**Alternativa con `while`:**

```
Node x = first;
while (x != null) {
    // procesamos x.item
    x = x.next;
}
```

**Nota:** Empezamos en `first` y seguimos los enlaces `next` hasta `null`

# Operaciones con listas: Insertar al principio

**Código:**

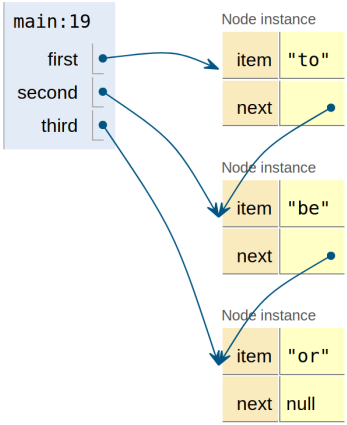
1. `Node oldfirst = first;`
2. `first = new Node();`
3. `first.item = "not";`
4. `first.next = oldfirst;`

**Complejidad:** O(1) - tiempo constante

**Ventaja:**

- No necesita recorrer la lista
- Solo actualiza referencia `first`

**Lista inicial:**




Inserta `"not"` al principio

# Operaciones con listas: Añadir al final

**Desafío:** ¿Cómo añadir un elemento al final de la lista?

**Situación:**

- Sólo disponemos del puntero `first`
- Queremos añadir un nuevo nodo al final
- El último nodo tiene `next = null`

**Ejercicio:**

*¿Cómo añadir un elemento al final de la lista enlazada?*

*Piensa en una solución antes de continuar...*

# Solución 1: Recorrer hasta el final

**Código para añadir al final (sin puntero `last`):**

```
Node last = first;
while (last.next != null) {
    last = last.next;
}
// Ahora creamos el nuevo nodo y lo enlazamos
last.next = new Node();
last.next.item = "not";
```

**Análisis:**

- Hay que recorrer toda la lista hasta encontrar el último nodo
- **Complejidad:** O(n) - proporcional al número de elementos
- **Problema:** Ineficiente para listas largas

# Solución 2: Mantener puntero a `last`

**Código para añadir al final (con puntero `last`):**

```
Node oldlast = last;     // guardamos el último nodo actual
last = new Node();       // creamos el nuevo nodo
last.item = "not";       // almacenamos el valor
last.next = null;        // nuevo nodo es el último
oldlast.next = last;     // enlazamos desde el anterior último
```

**Análisis:**

- No hay que recorrer la lista
- **Complejidad:** O(1) - tiempo constante
- **Coste:** Hay que mantener la variable `last` actualizada
- **Beneficio:** Operación muy eficiente

# Operaciones con listas: Añadir al final (resumen)

**Con referencia a `last`:**

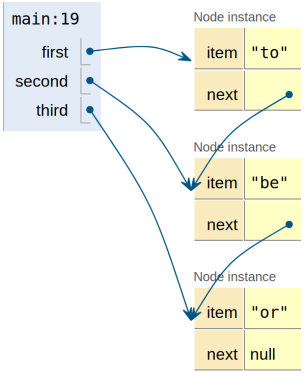
1. `Node oldlast = last;`
2. `last = new Node();`
3. `last.item = "not";`
4. `last.next = null;`
5. `oldlast.next = last;`

**Complejidad:** O(1) - tiempo constante

**Sin referencia a `last`:**

- Recorrer hasta el final: O(n)
- Menos eficiente

**Lista inicial:**




Añade `"not"` al final

# Operaciones con listas: Eliminar del principio

**Código:**

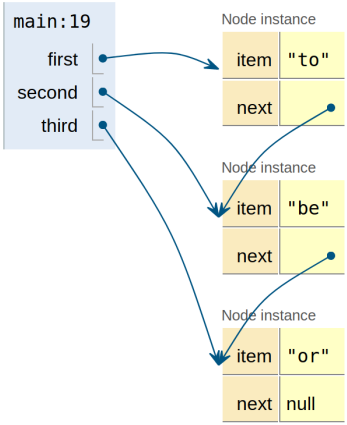
1. `Item item = first.item;`
2. `first = first.next;`
3. `return item;`

**Complejidad:** O(1) - tiempo constante

**Ventajas:**

- No necesita recorrer la lista
- El nodo eliminado será recuperado por GC

**Lista inicial:**




Elimina `"to"` del principio

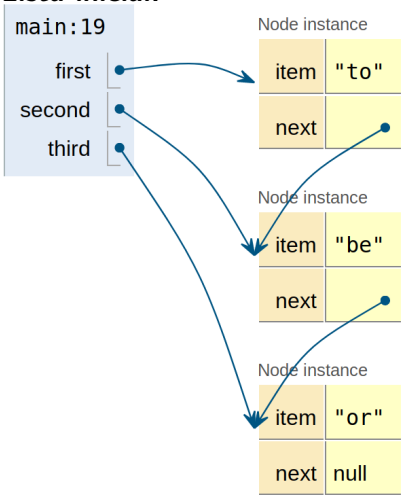
# Operaciones con listas: Eliminar al final

**Problema:** Necesitamos el penúltimo nodo
**Algoritmo:**

1. Recorrer hasta el penúltimo nodo
2. `Node current = first;`
3. `while (current.next.next != null)`
4. `  current = current.next;`
5. `Item item = current.next.item;`
6. `current.next = null;`
7. `last = current;` (si usamos `last`)

**Complejidad:** O(n) - recorre toda la lista

**Lista inicial:**



# Operaciones con listas: Eliminar nodo cualquiera

**Problema:** Eliminar al final es O(n)

**Solución:** Añadir referencia al nodo anterior

**Lista doblemente enlazada:**

- Cada nodo tiene `next` y `prev`
- `next` apunta al siguiente nodo
- `prev` apunta al nodo anterior
- Navegación bidireccional

**Ventajas:**

- Eliminar al final: O(1)
- Insertar antes de un nodo: O(1)
- Recorrer hacia atrás: O(n)

**Visualización:**

`null` $\leftarrow$ `[to]` $\leftrightarrow$ `[be]` $\leftrightarrow$ `[or]` $\rightarrow$ `null`

Cada nodo apunta al siguiente y al anterior

**Complejidad espacial:**

- 2 referencias por nodo
- vs. 1 en lista simple

# Listas doblemente enlazadas: Estructura del nodo

**Clase Node con `prev`:**

```
private class Node {
    Item item;
    Node next;
    Node prev;  // anterior
}
```

**Variables de instancia:**

```
private Node first;
private Node last;
private int n;
```

**Invariante:**

- `first.prev == null`
- `last.next == null`
- `x.next.prev == x`

**Características:**

- Doble enlace por nodo
- Navegación bidireccional
- Más memoria que lista simple
- Eliminar final: O(1)

**Trade-off:**

+ Operaciones más eficientes
− Mayor uso de memoria

# Listas doblemente enlazadas: Eliminar al final

**Operación:** Eliminar último nodo en O(1)

**Algoritmo:**

1. `Item item = last.item;`
2. `last = last.prev;`
3. Si `last != null`:
4. `  last.next = null;`
5. Si no: `first = null;`
6. `n--;`

**Complejidad:** O(1) - acceso directo

**Código:**

```
public Item removeLast() {
    if (isEmpty())
        throw new Exception();

    Item item = last.item;
    last = last.prev;

    if (last != null)
        last.next = null;
    else
        first = null;

    n--;
    return item;
}
```

**¡No recorre la lista!**

# Listas doblemente enlazadas: Operaciones adicionales

## Insertar al principio: O(1)

```
Node oldFirst = first;
first = new Node();
first.item = item;
first.next = oldFirst;
first.prev = null;
if (oldFirst != null)
    oldFirst.prev = first;
else
    last = first;
```

## Insertar al final: O(1)

```
Node oldLast = last;
last = new Node();
last.item = item;
last.prev = oldLast;
last.next = null;
if (oldLast != null)
    oldLast.next = last;
else
    first = last;
```

## Eliminar al principio: O(1)

```
Item item = first.item;
first = first.next;
if (first != null)
    first.prev = null;
else
    last = null;
return item;
```

## Comparación de complejidad:

| Operación | Simple | Doble |
|---|---|---|
| Insert inicio | O(1) | O(1) |
| Insert final | O(1)* | O(1) |
| Delete inicio | O(1) | O(1) |
| Delete final | O(n) | O(1) |

* Con referencia `last`

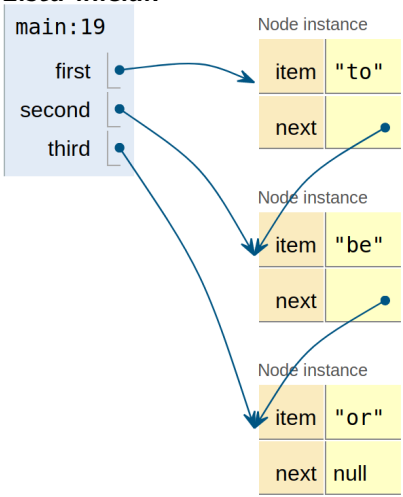
# Operaciones con listas: Eliminar nodo cualquiera

**Problema:** Eliminar nodo con valor específico
**Algoritmo:**

1. Buscar el nodo anterior al que se elimina
2. `Node current = first;`
3. `while (current.next != null)`
4. `  if (current.next.item.equals(x))`
5. `    current.next = current.next.next;`
6. `    return true;`
7. `  current = current.next;`
8. `return false;`

**Complejidad:** O(n) - buscar + eliminar

**Lista inicial:**



# Operaciones con listas: Insertar nodo cualquiera

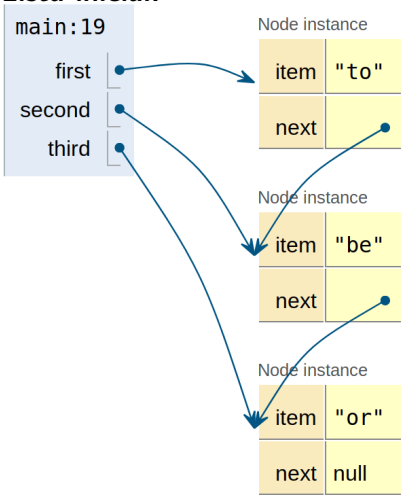
**Problema:** Insertar después de un nodo con valor `x`

**Algoritmo:**

1. Buscar el nodo con valor `x`
2. `Node current = first;`
3. `while (current != null)`
4. `  if (current.item.equals(x))`
5. `    Node newNode = new Node();`
6. `    newNode.item = item;`
7. `    newNode.next = current.next;`
8. `    current.next = newNode;`
9. `    return true;`
10. `  current = current.next;`

**Complejidad:** O(n) - buscar + insertar O(1)

**Lista inicial:**



# Operaciones con listas: Resumen

| **Operación** | **Complejidad** | **Requiere last?** |
|---|---|---|
| Insertar al principio | O(1) | No |
| Añadir al final | O(1) / O(n) | Sí / No |
| Eliminar del principio | O(1) | No |
| Eliminar del final | O(n) | No |
| Acceder por índice | O(n) | No |
| Buscar elemento | O(n) | No |

**Conclusiones:**

- Operaciones al principio son muy eficientes
- Añadir al final es eficiente con referencia `last`
- Eliminar del final requiere recorrer: O(n)
- No hay acceso aleatorio eficiente como en arrays

# Listas doblemente enlazadas

**Problema:** Eliminar al final es O(n)

**Solución:** Añadir referencia al nodo anterior

**Lista doblemente enlazada:**

- Cada nodo tiene `next` y `prev`
- `next` apunta al siguiente nodo
- `prev` apunta al nodo anterior
- Navegación bidireccional

**Ventajas:**

- Eliminar al final: O(1)
- Insertar antes de un nodo: O(1)
- Recorrer hacia atrás: O(n)

**Visualización:**

`null` $\leftarrow$ `[to]` $\leftrightarrow$ `[be]` $\leftrightarrow$ `[or]` $\rightarrow$ `null`

Cada nodo apunta al siguiente y al anterior

**Complejidad espacial:**

- 2 referencias por nodo
- vs. 1 en lista simple

# Listas doblemente enlazadas: Estructura del nodo

**Clase Node con `prev`:**

```
private class Node {
    Item item;
    Node next;
    Node prev;  // anterior
}
```

**Variables de instancia:**

```
private Node first;
private Node last;
private int n;
```

**Invariante:**

- `first.prev == null`
- `last.next == null`
- `x.next.prev == x`

**Características:**

- Doble enlace por nodo
- Navegación bidireccional
- Más memoria que lista simple
- Eliminar final: O(1)

**Trade-off:**

+ Operaciones más eficientes
− Mayor uso de memoria

# Listas doblemente enlazadas: Eliminar al final

**Operación:** Eliminar último nodo en O(1)

**Algoritmo:**

1. `Item item = last.item;`
2. `last = last.prev;`
3. Si `last != null`:
4. `  last.next = null;`
5. Si no: `first = null;`
6. `n--;`

**Complejidad:** O(1) - acceso directo

**Código:**

```
public Item removeLast() {
    if (isEmpty())
        throw new Exception();

    Item item = last.item;
    last = last.prev;

    if (last != null)
        last.next = null;
    else
        first = null;

    n--;
    return item;
}
```

**¡No recorre la lista!**

# Listas doblemente enlazadas: Operaciones adicionales

## Insertar al principio: O(1)

```
Node oldFirst = first;
first = new Node();
first.item = item;
first.next = oldFirst;
first.prev = null;
if (oldFirst != null)
    oldFirst.prev = first;
else
    last = first;
```

## Insertar al final: O(1)

```
Node oldLast = last;
last = new Node();
last.item = item;
last.prev = oldLast;
last.next = null;
if (oldLast != null)
    oldLast.next = last;
else
    first = last;
```

## Eliminar al principio: O(1)

```
Item item = first.item;
first = first.next;
if (first != null)
    first.prev = null;
else
    last = null;
return item;
```

## Comparación de complejidad:

| Operación | Simple | Doble |
|---|---|---|
| Insert inicio | O(1) | O(1) |
| Insert final | O(1)* | O(1) |
| Delete inicio | O(1) | O(1) |
| Delete final | O(n) | O(1) |

* Con referencia `last`

# Casos frontera: Lista vacía al añadir por el final

**Problema:** ¿Qué pasa cuando añadimos al final de una lista vacía?

**Código normal para añadir al final:**

```
Node oldlast = last;
last = new Node();
last.item = "not";
oldlast.next = last;
```

**Si la lista está vacía:**

- `first == null` y `last == null`
- Después del código: `last` apunta al nuevo nodo
- Pero `first` sigue siendo `null` **(ERROR)**

**Solución:** Detectar caso especial y actualizar `first`

```
if (first == null) first = last;
```

# Casos frontera: Eliminar cuando sólo hay un nodo

**Problema:** ¿Qué pasa cuando eliminamos el primer nodo y era el único?

**Código normal para eliminar del principio:**

```
Item item = first.item;
first = first.next;
```

**Si sólo había un nodo:**

- Después del código: `first == null` (correcto)
- Pero `last` sigue apuntando al nodo eliminado **(ERROR)**
- El nodo no puede ser recolectado por el garbage collector

**Solución:** Detectar caso especial y actualizar `last`

```
if (first == null) last = null;
```

# Casos frontera: Importancia

**¿Por qué son importantes los casos frontera?**

- Ocurren en situaciones límite (listas vacías, un solo elemento)
- Si no se manejan correctamente:
  - `NullPointerException` en tiempo de ejecución
  - Pérdidas de memoria (memory leaks)
  - Estado inconsistente de la estructura de datos
- Hay que identificarlos y tratarlos explícitamente
- Son comunes en implementaciones con punteros

# Stack con lista enlazada (1/2)

```
import java.util.Iterator;

public class Stack<Item> implements Iterable<Item>
{
    private Node first;  // cima de la pila
    private int n;       // número de elementos

    private class Node
    {
        Item item;
        Node next;
    }

    public boolean isEmpty() { return first == null; }
    public int size()        { return n; }
```

# Stack con lista enlazada (2/2)

```
    public void push(Item item)
    {
        Node oldfirst = first;
        first = new Node();
        first.item = item;
        first.next = oldfirst;
        n++;
    }

    public Item pop()
    {
        Item item = first.item;
        first = first.next;
        n--;
        return item;
    }
    // ... iterator() ...
}
```

# Iterador para lista enlazada

```
public Iterator<Item> iterator()
{
    return new ListIterator();
}

private class ListIterator implements Iterator<Item>
{
    private Node current = first;

    public boolean hasNext()
    {
        return current != null;
    }

    public Item next()
    {
        Item item = current.item;
        current = current.next;
        return item;
    }

    public void remove() { }
}
```

**Diferencia con array:** Recorre nodos, no índices

# Queue con lista enlazada: Decisiones

**Política FIFO:** Extraer del principio, añadir al final
**Variables de instancia:**

- `Node first`: primer elemento (más antiguo)
- `Node last`: último elemento (más reciente)
- `int n`: número de elementos

**Operaciones:**

- `enqueue()`: añade al final (`last`)
- `dequeue()`: extrae del principio (`first`)

**Estados frontera a considerar:**

- Cola vacía al añadir: `first` y `last` al mismo nodo
- Un solo elemento al extraer: `last = null`

# Queue con lista enlazada

```
public class Queue<Item> implements Iterable<Item>
{
    private Node first, last;
    private int n;

    private class Node { Item item; Node next; }

    public void enqueue(Item item)
    {
        Node oldlast = last;
        last = new Node();
        last.item = item;
        last.next = null;
        if (isEmpty()) first = last;  // caso frontera
        else           oldlast.next = last;
        n++;
    }

    public Item dequeue()
    {
        Item item = first.item;
        first = first.next;
        if (isEmpty()) last = null;   // caso frontera
        n--;
        return item;
    }
}
```

# Bag con lista enlazada

**Implementación simple:**

- Basada en Stack (código muy similar)
- Renombrar `push()` a `add()`
- Eliminar `pop()`
- Mantener iterador (orden no importa)

**Implementación alternativa:**

- Delegar en una instancia de Stack
- Composición en lugar de duplicar código
- Reutilizar código ya depurado

# Arrays vs Listas enlazadas

| Aspecto | Array | Lista enlazada |
|---|---|---|
| Acceso aleatorio | O(1) constante | O(n) secuencial |
| Insertar/eliminar inicio | O(n) | O(1) |
| Redimensionar | Copiar elementos | No necesario |
| Memoria extra | Capacidad no usada | Referencias |
| Localidad de caché | Buena | Regular |

**Conclusión:**

- Arrays: acceso aleatorio, memoria contigua
- Listas: inserción/eliminación eficiente, tamaño dinámico
- La elección depende del uso esperado

# Resumen de implementaciones

**Evolución de Stack:**

1. FixedCapacityStackOfStrings (básica, limitada)
2. FixedCapacityStack<Item> (genérica)
3. ResizingArrayStack<Item> (capacidad dinámica)
4. ResizingArrayStack<Item> + Iterator (iterable)
5. Stack<Item> con lista enlazada

**Lecciones aprendidas:**

- Genéricos: una implementación para todos los tipos
- Iteradores: abstracción de recorrido
- Estructura de datos afecta rendimiento
- Compromiso entre simplicidad y eficiencia

# Discusión

# Preguntas para discusión

- ¿Cuándo usar una pila en lugar de una cola?
- ¿Por qué es importante el soporte para iteradores?
- ¿Qué política de extracción es más común en aplicaciones reales?
- ¿Cómo afecta la representación de datos a la eficiencia?

# ¿Cuándo usar pila vs cola?

**Usar Pila (LIFO):**

- Cuando necesitas invertir el orden
- Navegación (historial del navegador)
- Evaluación de expresiones
- Análisis sintáctico (parsers)

**Usar Cola (FIFO):**

- Cuando el orden de llegada importa
- Colas de impresión
- Procesamiento de tareas
- Mantener orden de inserción
- ¡Mensajes en ROS 2!

# Importancia de los iteradores

**Sin iteradores:**

- Habría que exponer la implementación interna
- Código cliente dependiente de la estructura
- Dificulta cambiar la implementación

**Con iteradores:**

- Abstracción completa de la implementación
- Código cliente independiente
- Sintaxis foreach simple y clara
- Facilita mantenimiento y evolución

# Políticas de extracción en la práctica

**FIFO (Colas) más común:**

- Justicia: primero llegado, primero servido
- Sistemas operativos (scheduling)
- Redes (buffers de paquetes)
- Procesamiento de eventos

**LIFO (Pilas) en casos específicos:**

- Inversión de orden
- Análisis sintáctico
- Backtracking en algoritmos
- Deshacer/rehacer en editores

# Representación y eficiencia

**Arrays:**

- Acceso directo rápido (O(1))
- Tamaño fijo o redimensionamiento costoso
- Buena localidad de memoria

**Listas enlazadas:**

- Tamaño dinámico sin redimensionar
- Inserción/eliminación eficientes
- Acceso secuencial
- Más uso de memoria (referencias)

## Conclusión

La elección de representación impacta significativamente en el rendimiento